A Stochastic-Computing Tensor Unit and an RV32I Core on a 9K-LUT FPGA: Bit-Exact Co-Design for Tiny-MLP Inference

Author Name
Affiliation (placeholder)

***Abstract*—Stochastic computing (SC) replaces a multiplier by a single logic gate acting on random bit-streams, which makes it attractive for the smallest FPGAs, where dedicated multipliers are scarce. We present the co-design of an SC tensor processing unit (SC-TPU) and a multi-cycle RV32I processor for an 8640-LUT4 Gowin GW1NR-9 device (Sipeed Tang Nano 9K). The SC-TPU is a 16×8 weight-stationary tile of bipolar XNOR multipliers with accumulative parallel counters, driven by 8-step leap-forward LFSRs. Its behaviour, including the random-number sequence and the integer post-processing, is specified bit-exactly, so that a Python golden model, the training flow and the RTL must agree bit for bit; in simulation they do on 1454 inferences. For a 64-16-10 MLP on the 8×8 digits set the golden model gives 95.0 % test accuracy for the expected-value datapath and 94.4 % (95 % CI 91.6–96.4 %) for SC with independent per-lane generators at stream length 256 (92.8±1.4 % over 20 seed offsets). A seed revision that maximised the phase spacing of the generators made the operand-pair streams of different lanes time-translates of each other and lost 4.7 percentage points at stream length 512 (McNemar $p = 0.0015$); we derive two seed rules that exclude this and select seeds on validation data. The integrated SoC uses 65 % of the LUT4 positions and closes timing at 47.46 MHz (worst of three seeds) against the 27 MHz board clock; one inference takes 2928 cycles (108 µs). All results are simulations with an open-source flow; a bitstream was generated but never loaded into a device.**

***Index Terms*—stochastic computing, neural network accelerator, RISC-V, FPGA, LFSR seeding, bit-exact golden model, open-source FPGA toolchain**

I. Introduction

Low-cost FPGAs with fewer than ten thousand look-up tables are widely used in education, prototyping and small embedded products, but they offer only a handful of hard multipliers. The GW1NR-9 device used here has 8640 LUT4 positions, 6480 flip-flops, 26 block RAMs and 20 18×18 multipliers [1]. A binary int8 neural-network datapath on such a device is limited to at most 40 hard 9×9 multiply–accumulate (MAC) lanes, or must spend roughly 70 LUTs per soft 8×8 multiplier (rough CALCULATION).

Stochastic computing [2], [3] encodes a number as the probability of a one in a random bit-stream. A multiplication then reduces to a single AND (unipolar) or XNOR (bipolar) gate, and the precision becomes a run-time parameter: doubling the stream length L halves the variance of the result. Neural networks tolerate moderate arithmetic noise, which has made them a prominent application of SC [4]–[6]. The price is latency (L clock cycles per product), the cost of random-number generation, and a sensitivity to correlation between streams.

A second, less discussed difficulty is *verification*: an SC result is a random variable, so a hardware/software comparison is meaningless unless the random sequence is defined as precisely as the arithmetic. This paper treats the random-number generators, their seeding and their sequencing as part of a bit-exact architectural contract, and shows that the seeds themselves are a design parameter that needs its own rules and its own held-out evaluation.

Contributions.

1) A complete, small SoC for the Tang Nano 9K: a multi-cycle RV32I CPU [7] orchestrating a 16×8 weight-stationary bipolar SC tile with accumulative parallel counters (APC), implemented with an open-source flow [8]–[10] (Section III).
2) A bit-exact specification of the SC datapath (8-step leap-forward LFSR, three RNG decorrelation modes, integer post-processing), mirrored by a Python golden model used for noise-aware training and matched by the RTL in simulation (Section IV).
3) An accuracy study versus stream length and RNG mode with confidence intervals and a 20-offset seed ensemble, and an analytical variance model checked against simulation (Section V).
4) A documented design iteration on the LFSR seeds, including a negative result: phase-maximal seeds are *worse*; two seed rules and a pre-registered selection protocol replace them (Section V-C).

II. Background and Related Work

A. Stochastic number encoding

A stochastic number (SN) is a bit-stream $b_0, \ldots, b_{L-1}$. In *unipolar* encoding it represents $p = P(b_k{=}1) \in [0, 1]$; in *bipolar* encoding it represents $v = 2p - 1 \in [-1, 1]$, which admits signed weights with a single stream per operand [2], [3]. A stochastic number generator (SNG) converts a binary code c into a stream by comparing it with a pseudo-random number r: $b = [r < c]$. With r uniform on $\{0, \ldots, 2^n - 1\}$, $P(b{=}1) = c/2^n$.

B. Multiplication and accumulation

For two independent unipolar streams the AND gate gives $P(a \wedge b) = p_a p_b$. For bipolar streams the XNOR gate gives

$$P\big(\overline{a \oplus b} = 1\big) = \tfrac{1}{2}\,(1 + v_a v_b)\,, \tag{1}$$

i.e. the bipolar product [3], [4]. Accumulating many products with a multiplexer scales the sum down by the fan-in and loses precision. An accumulative parallel counter instead counts the ones of all product streams in each cycle and adds the count to a binary accumulator, so the sum is exact in expectation and leaves the stochastic domain; SC-DCNN [6] studied such APC-based inner products for deep convolutional networks. Kim *et al.* [5] exploited the stream length as a run-time energy–accuracy knob in deep networks.

C. Precision, correlation and LFSRs

An L-bit estimate of p has variance $p(1-p)/L$, so every factor of four in L adds one bit of precision. Equation (1) holds only for independent streams; streams generated from the same or from shifted random sequences are correlated and bias or inflate the error [3]. Linear-feedback shift registers (LFSRs) are the usual cheap random source. Taking n consecutive bits of a single-step LFSR as r yields successive values that are shifted copies of each other, and two LFSRs with the same polynomial produce lagged copies of the same maximal-length sequence. Both effects matter in a small design that cannot afford many independent generators.

D. Small RISC-V cores and open FPGA flows

PicoRV32 [11] is a size-optimised RV32 core with the same native valid/ready memory interface that we adopt. The Yosys and nextpnr tools [8], [9], together with the reverse-engineered Gowin database of Project Apicula [10], provide a fully open flow for the GW1N family; we use it exclusively.

III. Architecture

A. SoC overview

Fig. 1 shows the SoC. A single 27 MHz clock domain is driven by the board oscillator (no PLL); the reset button passes through a two-flip-flop synchroniser into a synchronous active-high reset. The CPU is the only bus master. Slaves are selected by address bits [31:28] (Table I); an access to an unmapped region returns zero, completes normally and sets a sticky debug flag, so a software error can never hang the bus. The bus is a native valid/ready protocol: the master holds `mem_valid`, address, data and byte strobes stable until the slave returns a one-cycle `mem_ready` pulse. RAM and peripherals answer in the second cycle, the SC-TPU in the third; a TPU read of its buffers while the controller owns their ports waits for up to 35 cycles (SIMULATION).

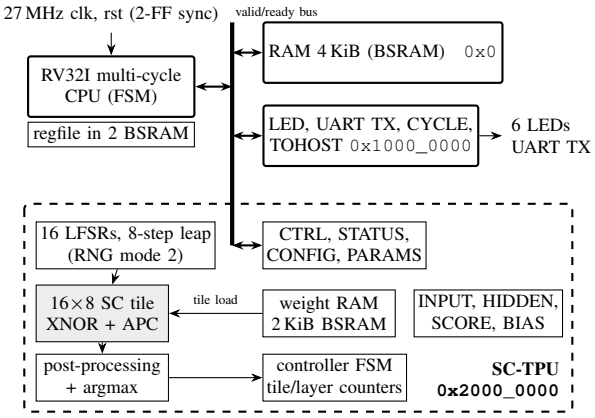


Fig. 1. SoC block diagram (architecture specification rev C). The CPU is the only bus master; the SC-TPU is a memory-mapped slave whose weight RAM is time-shared between CPU accesses and tile loads.

TABLE I
Memory map (spec rev C, source: architecture specification)

Base	Size	Content
`0x0000_0000`	4 KiB	RAM (program + data), reset PC = 0
`0x1000_0000`	32 B	LED, UART_TX/STATUS, CYCLE, TOHOST
`0x2000_0000`	4 KiB	SC-TPU: registers `0x000`–`0x017`, INPUT `0x040`, SCORE `0x080`, HIDDEN `0x0C0`, BIAS `0x100`, weight RAM `0x800`–`0xFFF`
other	–	read 0, write ignored, sticky `bus_err`

B. RV32I multi-cycle CPU

The CPU implements the RV32I base instruction set [7] with FENCE as a no-op; there are no CSRs and no interrupts. ECALL, EBREAK, illegal encodings (strictly decoded, including all CSR instructions, unknown funct3/funct7 values, `0x00000000` and `0xFFFFFFFF`) and misaligned accesses or jump targets halt the core with one of six trap causes. Because the CPU only orchestrates the accelerator, it is optimised for area:

- a state machine with the states CLEAR, FETCH, DECODE, EXECUTE, SHIFT or MEM, and WB; CLEAR zeroes the 32 registers once after reset (32 cycles), so the first fetch follows 31 cycles after reset release (SIMULATION);
- the register file is held in two block RAMs (one per read port), costing no LUT or flip-flop for its 1 kbit of state;
- a single 33-bit adder serves arithmetic, comparisons and address generation; branch targets are computed in DECODE so that the comparison in EXECUTE can reuse it;
- shifts are iterative (4 or 1 bit per cycle) instead of a barrel shifter.

The resulting cycles per instruction at zero wait states are (CALCULATION from the FSM): 3 for LUI, AUIPC, JAL, FENCE and branches, 4 for ALU operations, JALR and stores, 5 for loads, and $4 + \lfloor n/4 \rfloor + (n \bmod 4)$ for a shift by n.

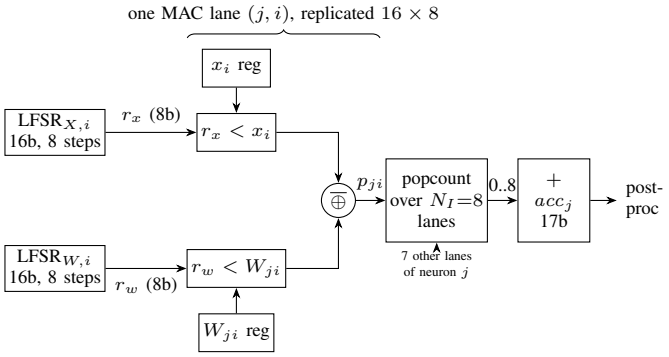


Fig. 2. One MAC lane of the SC tile and the APC of neuron j. The input comparator $[r_x < x_i]$ is shared by the 16 neuron lanes; every weight has its own comparator. In the default RNG mode 2 each input lane i has its own LFSR pair, shared by the 16 neuron lanes; modes 0/1 use one pair for all lanes.

C. SC-TPU dataflow

The workload is a 64-16-10 multilayer perceptron with a clipped-ReLU hidden layer and an argmax output. Every input, activation and weight is an 8-bit code c representing the bipolar value $v(c) = (c - 128)/128 \in [-1, 127/128]$. The array is a weight-stationary tile of $N_L = 16$ neuron lanes by $N_I = 8$ input lanes (Fig. 2). $N_L = 16$ equals the hidden width, so one pass of a layer-1 tile serves all hidden neurons. Layer 1 is processed in $T = 64/8 = 8$ tiles and layer 2 in $T = 16/8 = 2$ tiles; the six unused neuron lanes of layer 2 are padding and are ignored by the argmax. A fully parallel layer-1 array (1024 lanes, 8192 weight flip-flops) does not fit the 6480 flip-flops of the device, which fixed the tile choice.

For one tile the 128 weights W_{ji} and the 8 input codes x_i are loaded into registers; then, during $L = 2^K$ stream cycles k, every neuron lane updates its APC accumulator

$$acc_j \leftarrow acc_j + \sum_{i=0}^{N_I-1} \overline{[r_x(i,k) < x_i] \oplus [r_w(j,i,k) < W_{ji}]}. \tag{2}$$

The accumulators are cleared once per layer and keep running over its tiles, so after the last tile acc_j is the count of ones of the complete dot product.

D. Leap-forward LFSR and RNG modes

All generators are 16-bit Fibonacci LFSRs with the polynomial $x^{16} + x^{14} + x^{13} + x^{11} + 1$ (period 65535, maximal, CALCULATION). To avoid the shifted-copy correlation of consecutive single-step outputs, each stream cycle advances every LFSR by *eight* steps and uses the low byte of the new state as r (ADR-005). Each of the eight new bits is the XOR of four current-state bits, so the leap costs eight LUT4s and one logic level per LFSR (CALCULATION); the RTL registers the state one stream cycle ahead, so r is a flip-flop output. The golden model verifies the unrolled network against eight single steps for all 65536 states, and the RTL testbench does so for 70 000 cycles.

Three RNG modes are a synthesis parameter (ADR-003):

- *Mode 0 (shared)*: one LFSR$_X$ (seed `0xACE1`) for all input lanes and one LFSR$_W$ (seed `0xF9DE`, half a period later) for all weight lanes.
- *Mode 1 (rotate)*: the same two LFSRs, lane i uses $\mathrm{rotl}_8(r, i)$, a free re-wiring.
- *Mode 2 (independent, default)*: one LFSR pair per input lane, seeds $(\texttt{0xACE1} + \texttt{0x3C5A}\, i) \bmod 2^{16}$ and $(\texttt{0x7A5C}+\texttt{0x2B67}\, i) \bmod 2^{16}$, shared by the 16 neuron lanes.

The LFSRs are loaded with their seeds when START is accepted, advance only in stream cycles and are not reset between tiles or layers. In the RTL, mode 2 costs 111 LUT4 and 224 flip-flops (14 additional 16-bit LFSRs) more than mode 1 (SIMULATION of synthesis and place-and-route). Mode 1 was the default of the first specification (rev A); rev C makes mode 2 the default, with the seeds chosen as described in Section V-C.

E. Post-processing (bit-exact)

After the last tile of a layer each neuron j is post-processed with integer arithmetic exactly as in the specification, with N_{in} the layer fan-in, $K \in \{4, \ldots, 10\}$ and the activation shift $s \in \{0, \ldots, 7\}$:

$$D_j = 2\, acc_j - N_{in} L, \tag{3}$$

$$B_j = \left\lfloor bias_j\, 2^K / 128 \right\rfloor, \qquad D'_j = D_j + B_j, \tag{4}$$

$$u_j = \left\lfloor D'_j\, 2^{s+7} / 2^K \right\rfloor,$$

$$\mathrm{HIDDEN}_j = 128 + \mathrm{clip}(u_j, 0, 127), \tag{5}$$

$$\mathrm{SCORE}_j = D'_j \text{ (signed, 32 bits)}, \tag{6}$$

where (5) applies to layer 1 (hidden codes), (6) to layer 2, $bias_j$ is a signed 16-bit integer in units of $1/128$, and $\mathrm{RESULT} = \arg\max_{j \in \{0..9\}} \mathrm{SCORE}_j$ with ties resolved to the lowest index. Since $E[\overline{a \oplus b}] = (1 + v_a v_b)/2$, D'_j/L is an unbiased estimate of $\sum_i v(x_i) v(W_{ji}) + bias_j/128$ up to the floor in B_j. The RTL uses a 17-bit accumulator, 20-bit signed D, B and D', and a 34-bit exact shifter for $D'\, 2^{s+7}$, in a two-stage pipeline (CALCULATION, checked on 53 328 directed vectors in simulation).

F. Programming model and tile schedule

Weights and biases are written once through the bus into the weight RAM and the BIAS buffer (or pre-loaded from initialisation files in the bitstream). Per inference the CPU writes the 64 input codes (16 words), sets LOG2_L and ACT_SHIFT in CONFIG, writes START, polls STATUS.DONE and reads RESULT; the CYCLES register reports the busy time. Fig. 3 shows the tile schedule of the RTL controller. A tile load reads the 32 weight words from the block RAM in 32 consecutive cycles plus one cycle of read latency; the two input words are read in the same cycles. L stream cycles follow. After the last tile of a layer, 19 cycles ($N_L + 3$) read the biases, run the two-stage post-processing pipeline and update the argmax on the fly.

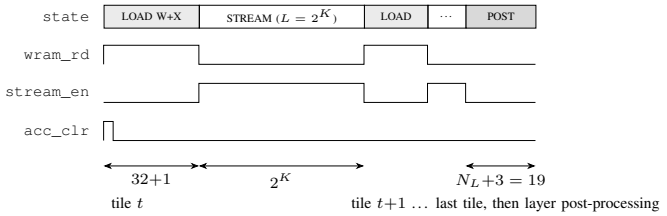


Fig. 3. Tile schedule of the RTL controller (durations in clock cycles; not to scale). The LFSRs advance only while `stream_en` is high. Ten tiles and two post-processing phases give CYCLES $= 10(2^K + 33) + 2 \cdot 19$ (CALCULATION, confirmed by SIMULATION for $K = 4..10$).

IV. Co-Design and Verification Methodology

A. Bit-exact golden model

The Python golden model implements the specification twice: a literal reference with explicit loops and eight single LFSR steps per stream cycle, and a vectorised engine that precomputes, per tile and lane, a count table from the histogram of (r_x, r_w) pairs, which makes the accuracy sweeps fast. All 593 exported test vectors (33 fields each: hidden codes, scores, result) were regenerated by the slow reference with 0 mismatches (SIMULATION); 158 unit tests pass. For the RTL the model exports, per RNG mode, the 360 test images at $K = 8$, 51 corner-case parameter sets, 63 sweep vectors ($K = 4..10$, $s \in \{0, 3, 7\}$) and cycle-by-cycle RNG traces. Every vector file carries a specification-revision header that the simulation scripts check. The RTL testbench compares all hidden codes, scores, RESULT and CYCLES of 1454 inferences over the three modes, plus 239 re-runs with out-of-range LOG2_L values, with 0 mismatching fields, and the per-lane random numbers of the first 64 stream cycles after every START with 0 mismatches (SIMULATION, Icarus Verilog 12.0 [12]).

B. Training with SC noise

The data set is the 8×8 handwritten-digits set distributed with scikit-learn [13], [14] (1797 images, pixels 0..16). A stratified split gives 1437 training and 360 test images; 288 of the training images form a validation split used for every model choice and for the seed selection, and the test split is used only for the final numbers. Pixels map to codes $c = 128 + \lfloor (127p + 8)/16 \rfloor$, i.e. non-negative bipolar values. Three variants were trained (Adam, 300 epochs, straight-through estimator for the code rounding): *float* (rounded after training), *qat* (quantisation-aware) and *qat_noise*, which adds zero-mean Gaussian noise with the independent-stream variance of (8) at $K = 8$ to both layers' pre-activations during training. The selected model is qat_noise with $s = 0$, chosen on the validation split; the final weights were then re-trained on all 1437 training images. The network was not re-trained for the later seed revisions.

C. Analytical error model

Writing each stream bit as a spin $\sigma = 2b - 1$ and assuming uniform, cycle-independent r and independent X and W generators, the variance of the estimate D'/L of one neuron is (CALCULATION)

$$\mathrm{Var}\left(\frac{D'}{L}\right) = \frac{1}{L} \sum_{\text{tiles}} \Big[\sum_{i,k} C_{ik}^x C_{ik}^w - \Big(\sum_i v_{x,i} v_{w,i} \Big)^2 \Big], \quad (7)$$

where C_{ik} is the spin correlation of lanes i and k within a tile ($C_{ii} = 1$). With shared numbers (mode 0) $C_{ik} = 1 - 2|a - b|/256$ for codes a, b; mode 1 is enumerated exactly over the 256 values of r; for independent lanes (mode 2) it reduces to

$$\mathrm{Var}(D'/L) = \frac{1}{L} \sum_i \left(1 - v_{x,i}^2 v_{w,i}^2\right). \quad (8)$$

The variance is largest for zero products; since most digit pixels are blank (code 128), layer 1 operates close to its worst case $64/L$. Sharing random numbers makes lanes with similar codes almost perfectly correlated and can multiply the variance by up to $N_I = 8$.

D. CPU verification by ISS-based differential testing

CPU stimuli are produced by a Python reference assembler and checked on a Python instruction-set simulator (ISS); they are testbench stimuli, not firmware. The suite has 44 programs: 11 directed, 20 random (200-instruction bodies, recorded seeds) and 13 trap tests covering every trap cause. Each program ends by dumping all registers into a 32-word signature. The ISS and assembler are covered by unit tests, including an assemble/disassemble round trip over 48 502 seeded instruction words with 0 mismatches. The self-checking RTL testbench runs every program under four bus timings (0, 1 and 3 wait states, and random 0–7 cycle stalls) and compares the signature, stop kind, trap cause, halt PC, retired-instruction count, TOHOST value, bus-error flag, LED and UART output with the ISS. A bus-protocol checker enforces seven rules (stable requests while waiting, no ready without valid, no X/Z, aligned fetches, exact byte strobes, one data access per instruction, silence after halt). The 44 programs are run again on the integrated SoC.

E. Checking the checkers

A passing testbench proves little unless it can fail. Each RTL testbench is therefore re-run with an injected fault and must report failure with a non-zero exit code: a wrong reference model in the TPU unit testbenches (LFSR tap, truncating instead of floor division, AND instead of XNOR), a flipped bit in an expected score (TPU and SoC), a corrupted expected CPU signature, a bus request that changes while waiting for `mem_ready`, a `mem_ready` without request, a wrong UART divider (240 instead of 234 clocks per bit) and an inverted LED polarity. All ten negative runs fail as required (SIMULATION). These are faults in the checkers, references and stimuli, not a systematic mutation campaign of the RTL.

F. Open-source implementation flow

Synthesis uses Yosys 0.70 with the Gowin script and wide-function multiplexers disabled (`-nowidelut`, chosen because it gave 502 fewer LUT4 positions and a 4.66 MHz faster

worst seed on the SoC); placement and routing use nextpnr-0.11.1 (himbaechel back end, family GW1N-9C, 27 MHz constraint), and the bitstream is packed with Apicula 0.32 only as a proof of routability; it was never programmed into a device. Simulation uses Icarus Verilog 12.0 [12] and lint Verilator 5.020 [15]. Each design is placed and routed with three seeds and the *worst* post-route Fmax is reported. The acceptance targets are Fmax $\geq$ 29.7 MHz (27 MHz + 10 %) and at most 85 % of each device resource.

V. Results

All results in this section are SIMULATION (golden model, RTL simulation or static timing analysis of the place-and-route result) or CALCULATION; none is a hardware measurement.

A. Accuracy versus stream length and RNG mode

Fig. 4 and Table II give the test accuracy of the bit-exact model with the deployed rev C seeds and, as a check that one seed set is not a lucky or unlucky draw, the mean over 20 analysis-only LFSR start offsets (all generators advanced by a common random number of steps, which keeps their relative lags). The reference is the expected-value (EV) model, in which every stream is replaced by its expectation: 95.0 % (342/360, CI 92.2–96.8 %). An exact int8 binary datapath on the same codes gives the identical result (0 mismatches in hidden codes, scores and results), so it serves as the non-SC baseline. A float-trained network evaluated in floating point reaches 96.1 % (346/360).

Four observations follow. First, at the reset default $K = 8$ ($L = 256$) the deployed seeds give 81.9, 89.2 and 94.4 % for modes 0, 1 and 2, but the ensemble means are 90.7, 92.8 and 92.8 %: much of the spread between the modes at a fixed seed set is realisation noise, and modes 1 and 2 have the same mean at $K = 8$. Second, shared numbers (mode 0) are clearly worse in both mean and spread: the ensemble standard deviation at $K = 8$ is 4.4 pp against 1.3–1.4 pp, and the deployed-seed accuracy is non-monotonic in K (89.7 % at $K = 7$, 81.9 % at $K = 8$), because with one generator pair all images see the same few empirical (r_x, r_w) histograms. Third, mode 2 has the smallest seed-to-seed spread at $K = 9$ (0.5 pp versus 0.8 and 1.1 pp). Fourth, as a design rule, the smallest K at which the deployed-seed accuracy stays within 1 and 2 percentage points (pp) of EV for every larger K is 10/9/8 for modes 0/1/2 (both thresholds); with the ensemble mean it is 10/9/9 within 1 pp and 9/9/9 within 2 pp. The release design therefore runs mode 2 at $K = 8$. One percentage point corresponds to 3.6 images, below the 95 % CI half-width of 2.3–3.1 pp for accuracies between 95 and 90 %; in particular the 2-image gap between mode 2 at $K = 8$ (340/360) and EV (342/360) is not statistically resolved, and the design rule is an engineering choice. Noise-aware training did not change the $s = 0$ result significantly (mode 2, $K = 8$: 95.3 % for qat versus 94.4 % for qat_noise), but post-training rounding of a float-trained model did not come within 1 pp of EV for any $K \leq 10$.

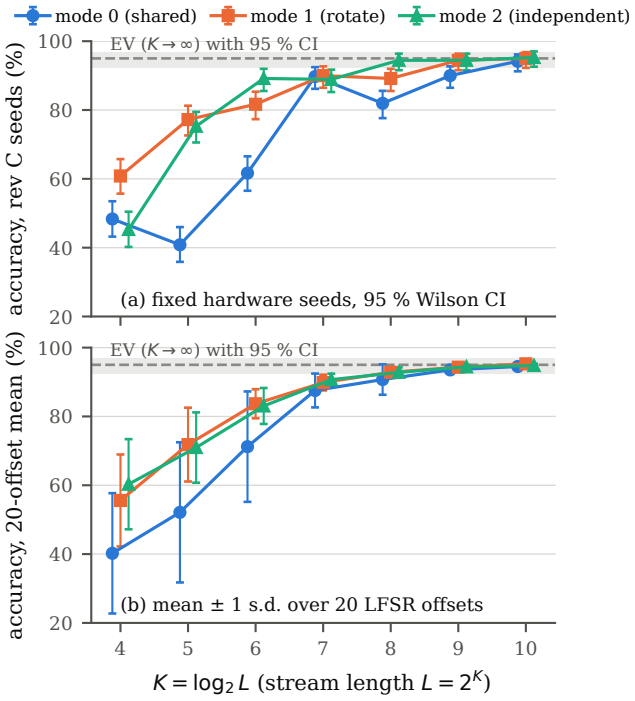


Fig. 4. Test accuracy ($n = 360$) of the bit-exact model versus K for the three RNG modes (SIMULATION). (a) Deployed rev C seeds with 95 % Wilson intervals [16]; (b) mean ± one standard deviation over 20 LFSR start offsets. Grey: EV model with its 95 % interval. Data: `reports/sw/accuracy_revC.json`.

B. Error model versus simulation

Fig. 5 compares the RMS error of D'/L (deterministic bias term removed) with (7). Over the 20-offset ensemble the ratio of measured to predicted RMS lies within 0.77–1.05 for all 42 layer/K/mode combinations; for mode 2 it lies within 0.90–1.05. The prediction tracks the $1/\sqrt{L}$ law and the penalty of mode 0 (at $K = 8$, layer 1: 0.70 measured, 0.79 predicted, versus 0.49/0.48 for mode 2). A single deployed-seed realisation can deviate much more, e.g. mode 0, layer 2, $K = 8$: 0.42 measured against 0.19 predicted. The model treats different stream cycles as independent. All LFSRs, however, run on the same maximal-length sequence: with the 8-step leap, two generators whose seeds are d single steps apart produce the same numbers with a lag of $d \cdot 8^{-1} \bmod 65535$ stream cycles. For the rev C mode-2 seeds the smallest such lag is 321 cycles (X_0/W_3), i.e. inside one layer from $K = 6$; such a lag couples only one operand of two products, and the ensemble agreement above indicates that its effect is small. No LFSR state repeats within an inference (at most 10 240 stream cycles), but the underlying bit sequence wraps after 8192 stream cycles, so at $K = 10$ a generator re-uses its own random numbers shifted by one bit; the $K = 10$ accuracy shows no penalty (CALCULATION).

TABLE II

TEST ACCURACY [%] VERSUS K (SIMULATION, GOLDEN MODEL, SELECTED MODEL QAT_NOISE, $s = 0$, $n = 360$). HW: DEPLOYED REV C SEEDS, CORRECT COUNT k AND 95 % WILSON CI; ENS.: MEAN ± S.D. OVER 20 LFSR START OFFSETS. EV ($K \to \infty$): 95.0 (342/360, CI 92.2–96.8).

K	Mode 0 (shared)			Mode 1 (rotate)			Mode 2 (independent, default)		
	HW (k)	CI	Ens.	HW (k)	CI	Ens.	HW (k)	CI	Ens.
4	48.3 (174)	43.2–53.5	40.2±17.5	60.8 (219)	55.7–65.7	55.6±13.4	45.3 (163)	40.2–50.4	60.3±13.1
5	40.8 (147)	35.9–46.0	52.1±20.4	77.2 (278)	72.6–81.3	71.8±10.7	75.3 (271)	70.6–79.5	71.0±10.2
6	61.7 (222)	56.5–66.5	71.2±16.0	81.7 (294)	77.3–85.3	83.7±4.2	89.2 (321)	85.5–92.0	83.0±5.2
7	89.7 (323)	86.2–92.5	87.5±4.9	90.0 (324)	86.5–92.7	89.8±2.3	88.9 (320)	85.2–91.7	90.7±1.7
8	81.9 (295)	77.6–85.6	90.7±4.4	89.2 (321)	85.5–92.0	92.8±1.3	94.4 (340)	91.6–96.4	92.8±1.4
9	90.0 (324)	86.5–92.7	93.5±1.1	94.4 (340)	91.6–96.4	94.3±0.8	94.4 (340)	91.6–96.4	94.4±0.5
10	94.2 (339)	91.2–96.2	94.5±0.7	95.0 (342)	92.2–96.8	95.3±0.5	95.3 (343)	92.6–97.0	94.8±0.5

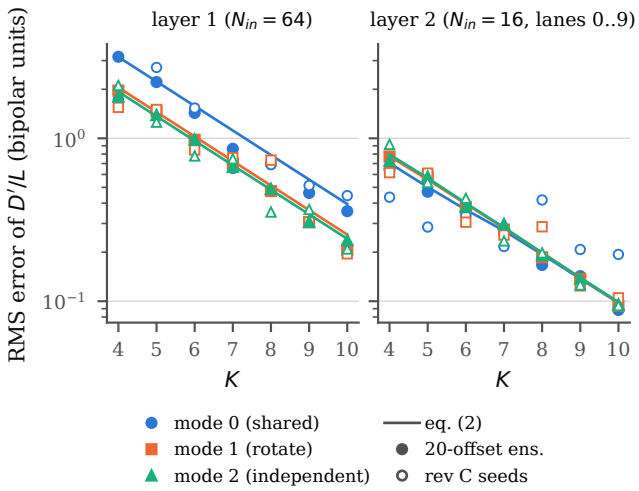


Fig. 5. RMS stochastic error of D'/L per neuron versus K (SIMULATION and CALCULATION): 20-offset ensemble (filled), prediction (7) (lines) and the single deployed rev C realisation (open). Layer 2 uses the SC hidden codes of the same run.

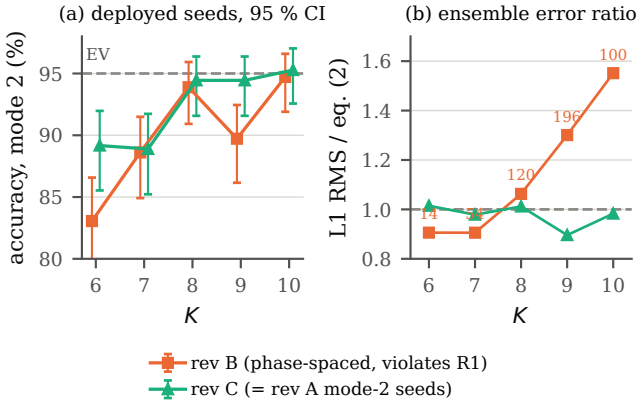


Fig. 6. Mode-2 seed iteration (SIMULATION, $n = 360$). (a) Deployed-seed accuracy with 95 % Wilson intervals, dashed: EV. (b) Layer-1 ensemble RMS error relative to eq. (7); numbers: identical count-table pairs of rev B (rev C: 0 at every K). Data: `reports/sw/revABC_comparison.json`.

C. Design iteration on the seeds: $A \to B \to C$

The first specification (rev A) defined the mode-2 seeds by the arithmetic progressions above without a structural argument. Because the lag analysis showed generators only 321 stream cycles apart, rev B re-derived them as maximally phase-spaced points of the common sequence, $X_i = S(4096\,i)$ and $W_i = S(32768{+}4096\,i)$ with $S(n)$ the state n single steps after `0xACE1`: any two of the 16 generators are then at least 512 stream cycles apart. This was a mistake. Since X_i and W_i advance by the *same* 4096 steps from lane to lane, the operand-pair stream (r_x, r_w) of lane i is that of lane $i{-}1$ delayed by 512 stream cycles. For $K \le 9$ this is a whole number of tiles, so identical XNOR count tables recur in different (tile, lane) slots (14, 54, 120, 196 and 100 slot pairs out of 3160 at $K = 6..10$; none for rev A), and their errors add coherently instead of averaging. Fig. 6 shows the consequence: at $K = 9$ mode 2 falls to 89.7 % (323/360) against 94.4 % (340/360), with 22 images correct only with the rev C seeds and 5 only with rev B (exact McNemar test [17], $p = 0.0015$; difference +4.7 pp, Newcombe 95 % interval [18] +2.0 to +7.8 pp). The measured layer-1 error exceeds (7) by 30 % and 55 % at $K = 9$ and 10.

The general requirement is not spacing but the absence of shared structure between the operand pairs of different lanes. With D_i the phase offset (in single steps) between W_i and X_i, we use two rules: **R1**, no two lanes' operand-pair streams are translates of each other ($D_i \neq D_j$ for $i \neq j$), and the stricter **R2**, no lane pair shares bits in both operands at a common lag, directly or swapped ($|D_i - D_j| > 14$ and $|D_i + D_j| > 14$, circularly; the window of ± 14 steps arises because r is an 8-bit window of one bit sequence). Rev B violates R1 on all 28 lane pairs. To select the rev C seeds without touching the test split, a protocol was fixed in advance: five candidates (C0 = rev A, C1 = rev B, C2 = rev B with the W lanes reversed, which satisfies R1 but not R2, and C3/C4 obtained by an exhaustive scan for the largest lags under R1 and R2) are evaluated on the 288 validation images, and the highest worst-case deployed accuracy over $K \in \{8, 9, 10\}$ wins. The ranking was C0 98.96 %, C3 98.26 %, C4 97.22 %, C2 95.83 %, C1 95.14 %. These images were part of the final training set, so the absolute values are optimistic and only the ranking is used; moreover C0, C3 and C4, which all satisfy R1 and R2, are not significantly different (paired McNemar $p \ge 0.125$).

TABLE III
INFERENCE LATENCY: PRE-RTL BUDGET (CALCULATION) AND RTL CYCLES (SIMULATION, EQ. (9)) AT 27 MHz, WITH THE DEPLOYED REV C ACCURACY (SIMULATION)

K	L	budget	RTL	time [µs]	inf./s	acc. m1 / m2 [%]
4	16	569	528	19.56	51 136	60.8 / 45.3
6	64	1049	1008	37.33	26 786	81.7 / 89.2
8	256	2969	2928	108.44	9 221	89.2 / 94.4
10	1024	10649	10608	392.89	2 545	95.0 / 95.3

The candidates that violate a rule are the two worst. Rev C therefore restores the rev A mode-2 seeds, which also carry the longest RTL verification history, now justified by an explicit rule and held-out evidence rather than by accident; on the test split rev C equals rev A in mode 2 image by image.

D. Latency and throughput

The RTL controller of Fig. 3 needs

$$\mathrm{CYCLES}(K) = 10 \cdot 2^K + 368 \text{ clock cycles} \qquad (9)$$

per inference: ten tiles of $2^K + 33$ cycles and two post-processing phases of 19 cycles (CALCULATION). The test-benches confirm it for every $K = 4..10$ and every vector (CYCLES register = counted busy cycles = formula, SIMULATION). The pre-RTL budget, $10 \cdot 2^K + 409$, had assumed 36 load cycles per tile and separate argmax and handshake cycles; the requirement allows at most 1.10 times the budget, i.e. 3265 cycles at $K = 8$. Table III pairs the latency with the accuracy of Table II. At $K = 8$ an inference takes 2928 cycles or 108.4 µs at 27 MHz (9221 inferences/s, 1.09×10^7 network MAC/s). On the integrated SoC the demonstration program, with weights, biases and one input image pre-loaded from the initialisation files, needs 2948 cycles from the START store to the RESULT load (CYCLES plus 20 cycles of polling and handshake) and 5441 cycles (201.5 µs) from reset release to completion, 981 instructions in total. The time for writing the 64 input codes was not simulated because this program does not write them.

E. CPU results

All 44 programs pass under all four bus timings (176/176 runs) with zero protocol violations, and both injected faults are detected (SIMULATION, Icarus Verilog 12.0). The aggregate CPI over the suite is 4.231 at zero wait states (28 206 cycles for 6667 instructions), 5.593 with one wait state (the block-RAM slave of the SoC), 8.317 with three and 9.013 with random stalls. Placed and routed alone with the 4 KiB RAM, the core closes at 62.46, 53.26 and 55.74 MHz for seeds 1–3 (SIMULATION, static timing analysis). The core maps to 831 LUTs, 74 carry cells, 111 flip-flops and two block RAMs; with the RAM and wrapper it occupies 957 of 8640 LUT4 positions (11 %). A first version with a barrel shifter needed 1212 LUT4 positions plus 463 wide-function multiplexers in the same context, which motivated the iterative shifter.

F. Resources and timing

Table IV collects the implementation figures. Before the RTL existed, a calibration benchmark of one 16×8 tile was synthesised and routed to size the design; the bottom-up SoC budget built on it was 5735 LUT4 (66.4 %) and 2489 flip-flops (CALCULATION). The integrated SoC needs 5628 LUT4 positions (65 %), 2803 flip-flops (43 %), 5 of 26 block RAMs (program RAM 2, weight RAM 1, CPU register file 2) and no hard multiplier; every resource stays below the 85 % limit. The total matches the budget within 2 % only by compensation: the TPU came out 20 % larger and the CPU 51 % smaller than budgeted. The worst of three seeds closes at 47.46 MHz (51.98, 47.46 and 51.21 MHz), 59.8 % above the 29.7 MHz target. The critical path (21.07 ns, of which 12.24 ns routing) is not the SC datapath, as the budget had predicted, but the CPU: instruction register, decoder and next-state logic, ALU operand selection, the 32-bit carry chain and the program counter. The SC-TPU alone reaches 60.43 MHz. Of the geometry variants, $N_I = 4$ fits easily but needs 1.9 times the cycles (5498 at $K = 8$), while $N_I = 16$ (73–77 % LUT4, carry-heavy) found no legal placement on any attempted seed, which shows that the practical placement limit for such a design can lie below the 85 % rule.

VI. DISCUSSION AND LIMITATIONS

SC versus binary arithmetic on this device. The SC tile provides 128 bit-level MAC lanes without using any of the 20 hard multipliers, and its precision is chosen at run time through CONFIG. Its throughput, however, is low: at $K = 8$ the rate is 1.09×10^7 useful MAC/s, whereas the 40 9×9 multipliers of the device clocked at 27 MHz could in principle deliver 1.08×10^9 int8 MAC/s (CALCULATION, no such design was built). On this device SC is therefore a study of area, decorrelation and verification, not a throughput win; its appeal grows where hard multipliers are absent or energy per operation dominates, which we have not measured.

Seeds are a design parameter. The rev B episode shows that an intuitive criterion (maximal phase distance) can be worse than an arbitrary choice, and that the failure is invisible at the default $K = 8$ (93.9 versus 94.4 %, McNemar $p = 0.79$) and appears only at $K = 9$. Seed sets should be checked against structural rules such as R1/R2, compared at several K, and selected on data that are not used for the final claim. Our validation images had been seen in training, so the selection is weaker than a truly held-out one would be.

Simulation only. No bitstream has been loaded into a device and no quantity in this paper is a hardware measurement. Fmax values come from the static timing model of nextpnr, whose Gowin delay data are reverse-engineered and not validated against silicon; no gate-level simulation was run because no behavioural models of the Gowin block RAMs are available in the flow. No power figure is reported: no open power model exists for the GW1N family. A physical validation plan (LED and UART output after reset, cycle counts, supply current, reset behaviour) is part of the engineering report but has not been executed.

TABLE IV

RESOURCES (GW1NR-9: 8640 LUT4, 6480 FF, 26 BSRAM) AND WORST-SEED FMAX (SIMULATION: SYNTHESIS, PLACE-AND-ROUTE AND STATIC TIMING; THREE SEEDS UNLESS NOTED). LUT4 COUNTS INCLUDE CARRY CELLS AND LUT-RAM POSITIONS. FLOW `-NOWIDELUT` UNLESS NOTED.

Block	LUT4	FF	BSRAM	Fmax [MHz]	Basis
CPU + 4 KiB RAM	957	126	4	53.26	RTL, stand-alone wrapper
SC tile bench, mode 1 / mode 2	2855 / 2967	1454 / 1678	0	52.12 / 52.98	pre-RTL calibration bench
SoC budget	5735	2489	3	–	CALCULATION (bottom-up, mode 1)
SC-TPU, $N_I = 8$, mode 2	4572	2669	1	60.43	RTL, stand-alone wrapper[a]
SC-TPU, $N_I = 4$, mode 1	3542	1901	1	65.30	RTL variant[a]
SC-TPU, $N_I = 16$, mode 1	6322	3533	1	no placement	RTL variant[a] (1 seed)
SoC, rev C	**5628**	**2803**	**5**	**47.46**	release build
SoC, rev C, default flow	6130	2803	5	42.80	flow comparison

[a] Synthesised from the rev A RTL: mode-2 seeds identical to rev C; only the PARAMS version constant and the modes 0/1 W seed differ.

Statistical resolution. With 360 test images one image is 0.28 pp and the 95 % interval half-width is 2.3–3.1 pp for accuracies between 95 and 90 %. Differences of 1–2 pp between modes or between $K = 8$ and $K = 9$ are not resolved, and the selected operating point should be read as an engineering choice. Paired tests on the same images (McNemar) resolve larger effects such as the rev B defect.

Workload, interfaces and data sources. The 64-16-10 MLP on 8×8 digits is a deliberately tiny workload; conclusions about accuracy versus K may not transfer to larger networks. The demonstration does not measure the CPU-side cost of loading inputs. The Gowin data sheet and memory user guide could not be opened during this work; device totals were taken from the board vendor's documentation and the open-source chip database, which agree, while the block-RAM organisation and the mapping of the two board buttons to pins remain assumptions.

Independent review. An independent review of the RTL, the golden model and this article was in progress when this revision was written; its findings are not yet reflected here.

VII. CONCLUSION

We have specified, modelled, implemented and verified in simulation a stochastic-computing tensor unit and an RV32I processor for a 9K-LUT FPGA with a fully open-source flow. Treating the random-number generators as part of a bit-exact contract made the SC datapath verifiable against a golden model (0 mismatching fields over 1454 inferences) and exposed the role of decorrelation and of the seeds themselves: in simulation, independent per-lane generators reach 94.4 % at $L = 256$, within the confidence interval of the 95.0 % expected-value accuracy, whereas a phase-maximal seed set lost 4.7 pp at $L = 512$ because it made different lanes' operand-pair streams time-translates. A closed-form variance model predicts the ensemble error within a factor 0.77–1.05 and supports the stream-length design rule. The SoC uses 65 % of the device and closes timing at 47.46 MHz against a 27 MHz clock. Measurements on hardware remain to be done.

REFERENCES

[1] Sipeed, "Tang Nano 9K," Sipeed Wiki (source file https://github.com/sipeed/sipeed_wiki, docs/hardware/en/tang/Tang-Nano-9K/Nano-9K.md), accessed 2026-11-04.

[2] B. R. Gaines, "Stochastic computing systems," in *Advances in Information Systems Science*, J. T. Tou, Ed. New York, NY, USA: Plenum Press, 1969, vol. 2.

[3] A. Alaghi and J. P. Hayes, "Survey of stochastic computing," *ACM Transactions on Embedded Computing Systems*, vol. 12, no. 2s, 2013.

[4] B. D. Brown and H. C. Card, "Stochastic neural computation I: Computational elements," *IEEE Transactions on Computers*, vol. 50, no. 9, 2001.

[5] K. Kim *et al.*, "Dynamic energy-accuracy trade-off using stochastic computing in deep neural networks," in *Proceedings of the 53rd Annual Design Automation Conference (DAC)*, 2016.

[6] A. Ren *et al.*, "SC-DCNN: Highly-scalable deep convolutional neural network using stochastic computing," in *Proceedings of the 22nd International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2017.

[7] A. Waterman and K. Asanović, *The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA*, RISC-V Foundation, 2019, document version 20191213.

[8] D. Shah *et al.*, "Yosys+nextpnr: An open source framework from Verilog to bitstream for commercial FPGAs," in *Proceedings of the IEEE International Symposium on Field-Programmable Custom Computing Machines (FCCM)*, 2019.

[9] C. Wolf, "Yosys open SYnthesis suite," https://github.com/YosysHQ/yosys, version 0.70 (YoWASP build) used in this work.

[10] "Project Apicula: open-source bitstream documentation and tools for Gowin FPGAs," https://github.com/YosysHQ/apicula, version 0.32 used in this work.

[11] C. Wolf, "PicoRV32 – a size-optimized RISC-V CPU," GitHub repository, https://github.com/YosysHQ/picorv32, commit ef203c2 used as a calibration proxy.

[12] S. Williams, "Icarus Verilog," https://github.com/steveicarus/iverilog, version 12.0 used in this work.

[13] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[14] E. Alpaydin and C. Kaynak, "Optical recognition of handwritten digits," UCI Machine Learning Repository, 1998.

[15] W. Snyder, "Verilator," https://www.veripool.org/verilator/, version 5.020 used in this work.

[16] E. B. Wilson, "Probable inference, the law of succession, and statistical inference," *Journal of the American Statistical Association*, vol. 22, no. 158, pp. 209–212, 1927.

[17] Q. McNemar, "Note on the sampling error of the difference between correlated proportions or percentages," *Psychometrika*, vol. 12, no. 2, pp. 153–157, 1947.

[18] R. G. Newcombe, "Improved confidence intervals for the difference between binomial proportions based on paired data," *Statistics in Medicine*, vol. 17, no. 22, pp. 2635–2650, 1998.